

Computer Networks

NST-CS-CN503 | Semester 5 • 2026 | Complete Visual Cheatsheet & Revision Compendium

TOTAL LECTURES
14 Comprehensive Units

TOTAL PAGES
15 Pages (Cover + Notes)

COURSE TRACK
Semester 5 Examination Prep

TABLE OF CONTENTS & INDEX

UNIT	LECTURE TOPIC & SUMMARY	PAGE
L-01	Introduction: The Cloud, Infrastructure & Layers	Page 02
L-02	Moving Data Through the Core: Switching & Delays	Page 03
L-03	The OSI and TCP/IP Reference Models	Page 04
L-04	Networking Devices, Topologies & The Cloud Network	Page 05
L-05	Application Layer: Architecture, Protocols & APIs	Page 06
L-06	HTTPS & TLS Handshake: Securing the Web	Page 07
L-07	Electronic Mail Protocols: SMTP, POP3, IMAP & MX	Page 08
L-08	The Socket API & Network Programming	Page 09
L-09	Transport Layer Principles & User Datagram Protocol	Page 10
L-10	Principles of Reliable Data Transfer (rdt)	Page 11
L-11	TCP Congestion Control & Flow Control	Page 12
L-12	Cloud Load Balancing & Traffic Management	Page 13
L-13	Addressing the World: IPv4 & Classless CIDR Routing	Page 14
L-14	Subnetting in Practice & NAT Architecture	Page 15

COMPUTER NETWORKS FOUNDATIONS

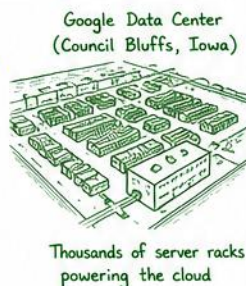
Lecture 01 : Introduction
The Cloud, Infrastructure & Top-Down Layers

← Back to Dashboard | Open PDF in New Tab →

1 DEMYSTIFYING THE CLOUD & GLOBAL INFRASTRUCTURE

1.1 What is the Cloud?

- "The cloud is just someone else's computer." → Major simplification.
- Cloud = massive, interconnected network of physical computers (servers) in data centers worldwide.
- Example: Google's data center, Council Bluffs, Iowa → thousands of servers working together.
→ All machines communicate instantaneously.



1.2 Cloud Service Network Cards

- Providers: AWS, Microsoft Azure, GCP
- Rely on ultra-fast network infrastructure to move massive data efficiently.
- Use specialized hardware like high-speed NICs (Network Interface Cards) to interconnect GPUs & handle complex workloads.



THE NETWORKING MISSION

→ Deliver digital packets (0s & 1s) from source to destination as quickly, reliably & securely as possible.

2 NETWORK CLASSIFICATIONS (PAN, LAN, MAN, WAN)

2.1 Traditional Network Types

- PAN**
(Personal Area Network)
Very short range (few meters).

Ex: smartphone ↔ wireless earphones, smartwatch, Bluetooth accessories



- LAN**
(Local Area Network)
Limited area: home, school, office, building.

Ex: Connecting local computers & devices to share local resources.



- MAN**
(Metropolitan Area Network)
Spans a city or large campus. Operated by municipality, ISP or large corporations.

Ex: Linking multiple LANs across a city.



- WAN**
(Wide Area Network)
Covers countries/continents. Links distant branch offices. Backbone of the Internet.



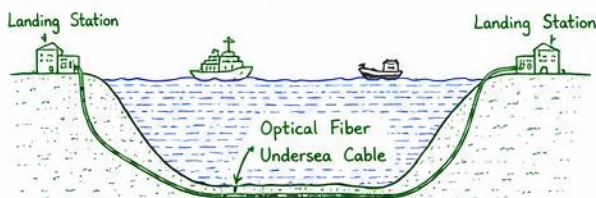
2.2 Network Boundaries & Scopes

Modern networks blur these lines, but classification helps in analyzing security policies, propagation delays & routing architectures.

3 UNDERSEA CABLES & PHYSICAL SECURITY CHALLENGES

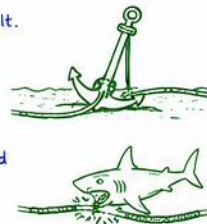
3.1 Subsea Physical Cable Infrastructure

- Internet depends on physical infrastructure.
- Data travels across continents via optical fiber cables laid on the seabed.
- As of 2025, there are 597+ undersea cable systems carrying bulk of global internet traffic.



3.2 Real-world Cable Hazards

- Keeping subsea networks safe is extremely difficult.
- Fishing Anchors:** Leading cause of transatlantic cable damage. Anchors dragged on seafloor can snag & rupture fiber lines.
- Marine Life (Sharks):** Sharks have been observed biting cables → operators shield transatlantic cables with steel-reinforced protective sleeves.



INFRASTRUCTURE VULNERABILITY

A single cut in a primary undersea fiber line can disrupt internet service for entire countries → requires specialized repair ships to locate, retrieve & splice broken fibers deep underwater.

4 THE TOP-DOWN LAYERED APPROACH

4.1 Why Start with Applications?

- When you open Instagram, tap an icon, and a video plays in fraction of a second → what happens in between?
- Users think: "Wi-Fi to Instagram".
- The hidden complex sequence = what we study in this course.

TOP-DOWN APPROACH

What you see/use → Requirements first → Mechanisms later

4.2 The Five-Layer OSI / TCP-IP Model Stack

Layer	Purpose	Examples / Protocols
⑤ Application	Network services to applications (what users see)	HTTP, HTTPS, DNS, Email (SMTP), Streaming, FTP
④ Transport	End-to-end process communication (reliability / ports)	TCP (reliable), UDP (fast), Ports (e.g., 80, 443)
③ Network	Logical addressing & routing across networks	IP (IPv4/IPv6), Routing (OSPF, BGP, RIP)
② Link	Local delivery between adjacent devices	Ethernet, MAC addr, Switching, ARP, Frames
① Physical	Transmission of raw bits over physical medium	Copper, Fiber, Radio, Signal modulation

WHY TOP-DOWN?

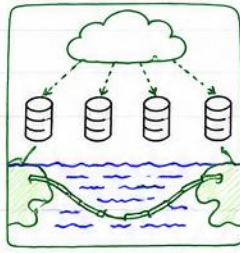
- ✓ Start with applications (e.g., HTTP).
- ✓ Understand **WHY** requirements exist (reliability, speed, security, etc.).
- ✓ Then study **HOW** each lower layer fulfills those requirements.

This makes concepts easier, logical & meaningful!

1 GLOBAL INFRASTRUCTURE BECAP & THE AI NETWORK ERA

1.1 Core Components & Scale

Global Infrastructure



- **Physical Servers:** Millions of servers are housed inside data centers worldwide.
- **Subsea Cabling:** Over **597** submarine cables route data globally across oceans, connecting continents dynamically.

1.2 AI Network Requirements

- Immense Compute Needs: Training LLMs requires clustering **10,000+ GPUs**.
 - Ultra-low Latency: Standard networks are too slow for fast GPU-to-GPU communications.
- ⇒ **AI requires specialized, high-bandwidth interconnects to avoid networking bottlenecks during parallelized matrix calculations.**

★ Key Insight

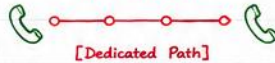
The network is just as critical as the compute hardware in the modern AI training stack. A slow connection will leave expensive GPUs sitting idle, waiting for parameter weights to sync.

2 CIRCUIT SWITCHING vs. PACKET SWITCHING

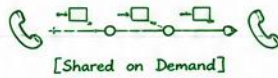
2.1 Circuit Switching: Dedicated Paths

- Classic landline telephone network model.
- Before communication, source & destination establish a dedicated end-to-end path (circuit).
- **Guaranteed Capacity:** Once set up, capacity reserved exclusively → predictable, stable rate with zero surprises.
- **Inefficient for Bursty Data:** If users go silent (e.g., 10 sec to think), the reserved capacity sits idle → cannot be used by others.

Circuit Switching (Landline Call)



Packet Switching (WhatsApp Call)



2.2 Packet Switching: Sharing on Demand

- No reservations. Resources allocated on demand.
- **Idle Efficiency:** If you stop talking or pause, zero packets are pushed → capacity is instantly available for others.
- **Resource Sharing:** Many users share the same physical links dynamically.

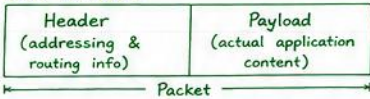
Analogy: The Landline vs. WhatsApp Call

- **Landline (Circuit Switching):** When Grandma pauses for 10 sec, that line's resource sits idle, reserved, but wasted.
- **WhatsApp (Packet Switching):** When you pause for 10 sec, no packets are sent. Other users' packets fill the space, maximizing utility of the link.

3 PACKETS, STORE-AND-FORWARD & BEST-EFFORT CORE

3.1 What is a Packet? & Store-and-Forward

- In packet-switched networks, applications slice data into small chunks called **packets**.
- **Packet Structure:**



- **Store-and-Forward Rule:** Most routers must receive & store the entire packet before transmitting the first bit on the next hop → allows header check & error verification.

3.2 Statistical Multiplexing & Best-Effort Delivery

- Instead of static partitioning (like circuit switching), packet switching aggregates bursty traffic dynamically → **Statistical Multiplexing**.
- **Superpower 1 (Efficiency):** Not all users transmit at max rates simultaneously → more active users share a single link.
- **Superpower 2 (Resilience):** Packets do not rely on pre-reserved static paths. If a router/link fails, packets route dynamically around the failure spot.
- **The Trade-off (Best-Effort Core):** No built-in performance guarantees. Packets can wait in queues, experience delay variance (jitter), or be dropped if buffer overflows.

Why TCP Exists

Because the network core is a "best-effort", unreliable packet-switching environment, protocols like TCP are needed at the edges. TCP handles error recovery, retransmits lost packets, and manages flow control to present a reliable interface to applications.

4 THE FOUR SOURCES OF NETWORK DELAY

4.1 Processing & Queuing Delay

1. **Nodal Processing Delay (d_{proc}):** Time to evaluate header, verify checksum (bit errors), and determine outgoing interface. (Typically microseconds, tiny).
2. **Queuing Delay (d_{queue}):** Time packet spends waiting in router's buffer before being pushed onto the link. Highly variable.



4.2 Transmission vs. Propagation Delay

3. **Transmission Delay (d_{trans}):** Time to push all packet bits onto the link. Depends on packet length (L bits) and rate (R bits/sec).

$$d_{trans} = \frac{L}{R}$$

4. **Propagation Delay (d_{prop}):** Time for a single bit to physically travel across the link. Depends on distance (d) and wave speed (s) ($\approx 2 \times 10^8$ m/s in optical fiber).

$$d_{prop} = \frac{d}{s}$$

4.3 Caravan Analogy & Numerical Examples

Caravan Analogy

-
- **Transmission Delay:** Time toll booth pushes cars one by one onto highway. ↑ Link rate (more booths) → ↓ delay.
 - **Propagation Delay:** Time car drives down road at speed limit to next booth. ↑ Lanes ≠ faster drive, only speed/distance affect.

Worked Numerical Example: Transmission Delay

Packet size $L = 8,000$ bits, Link rate $R = 2$ Mbps.

$$d_{trans} = \frac{8000}{2,000,000} = 0.004 \text{ sec} = 4 \text{ ms}$$

Scaling Link Rate (R)

Rate (Mbps)	2	4	8
d_{trans} (ms)	4	2	1

Physical Propagation Delay: Geostationary Satellite Link

- Satellite altitude $\approx 35,786$ km. Round trip distance: $d \approx 2 \times 35,786 \text{ km} = 71,572 \text{ km} = 71,572,000 \text{ m}$
- Speed of light (vacuum) $= 3 \times 10^8 \text{ m/s}$
- One-way propagation delay:

$$d_{prop} = \frac{71,572,000}{300,000,000} \approx 238.5 \text{ ms}$$

Bandwidth (500 Mbps vs 1 Gbps) does NOT reduce this delay.

Round-trip delay $\approx 477 \text{ ms}$
(~ half a second)
→ Unavoidable by physics!

5 THROUGHPUT & BOTTLENECK LINKS

5.1 Understanding Throughput

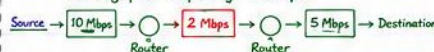
- **Throughput:** Actual rate (bits/sec) at which data is successfully transferred from source to destination over the network. Real-world performance, not theoretical maximum.

$$\text{Throughput} = \frac{\text{Useful Bits Delivered}}{\text{Time Taken}}$$

5.2 Finding the Bottleneck Hop

- End-to-end throughput is constrained by the slowest link on the path → **Bottleneck Link**.

Throughput Capacity Example



Bottleneck = 2 Mbps ⇒ End-to-end Throughput = 2 Mbps

5.3 Case Study: "I pay for 300 Mbps. Why is my video buffering?"

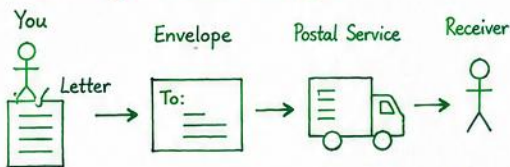
- Bottleneck could be hiding anywhere along the path:

 1. Overloaded video server (e.g., Netflix server capacity exhausted).
 2. Congestion on a transit link between your ISP and the content provider.
 3. Local interference on your home Wi-Fi network (walls, distance, other devices).

Your 300 Mbps plan = max capacity of your access link (just one hop in a long path).

1. THE IDEA BEHIND LAYERING

1.1 Sending a Letter Analogy



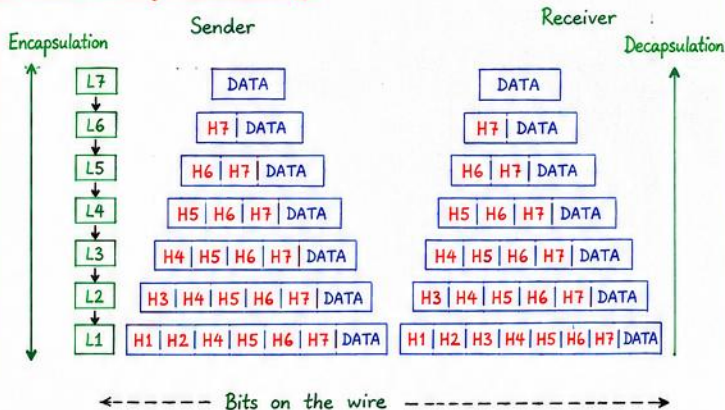
- Mail carrier only reads address (outside).
- Does NOT read contents (inside).
- If it reads, **privacy breaks & layering fails!**

1.2 What Layering Buys Us

- Isolation of Problems: Each layer solves one job.
- Modularity & Upgradability: Replace a layer e.g., Wi-Fi ↔ Ethernet (apps unchanged).
- Interoperability: Vendors build components separately if interfaces are fixed.
- Localising Faults: Easier debugging (ex: "Layer 4 issue").

2. ENCAPSULATION & DECAPSULATION

1.1 The Nesting Doll Analogy



Key Idea:

Layer N on receiver reads exactly the header written by Layer N on sender.
No layer touches other layers' headers.

1.2 Gift-Wrapping Analogy



Wrap going out
↓
Unwrap in reverse order on receipt.
Can't untie ribbon last!

3. THE 7-LAYER OSI MODEL

1.1 Open Systems Interconnection Standard

- OSI = Open Systems Interconnection.
- 7-layer model by ISO to ensure interoperation between different vendors.

1.2 Importance & Memory Aids

- Industry's shared language for troubleshooting: (ex: "Layer 4 issue").

- Mnemonics:

Bottom-up (L1 → L7):

Please Do Not Throw Sausage Pizza Away

Top-down (L7 → L1):

All People Seem To Need Data Processing

7	Application	User/App Layers
6	Presentation	
5	Session	Middle Layers
4	Transport	
3	Network	Lower Layers
2	Data Link	
1	Physical	Bit Delivery

4. SEVEN LAYERS, ONE AT A TIME

⑦	Application Layer Delivers network services to user/apps (web, email, FTP). Unit: Data	Analogy: Waiter
⑥	Presentation Layer Formats, encrypts, compresses, translates data representation.	Analogy: Translator
⑤	Session Layer Establishes, manages, terminates sessions. Dialog control, sync.	Analogy: Moderator
④	Transport Layer End-to-end delivery. Segmentation, reassembly, multiplexing, flow & error control (TCP/UDP).	Analogy: Phone Line
③	Network Layer Logical addressing (IP) & routing packets across multiple networks.	Analogy: GPS System
②	Data Link Layer Reliable transfer over a single link. MAC addressing, media access control, error detection (frames).	Analogy: Traffic Controller
①	Physical Layer Transmits raw bits (0s & 1s) over physical media (copper, fiber, radio). Physical signals & bit timing.	Analogy: Physical Roads

5. THE TCP/IP MODEL & COMPARISON

1.1 TCP/IP Protocol Stack Overview

- Working model of the Internet (developed by DARPA).
- 4 layers (collapses OSI's 7 layers).
- OSI = conceptual & for teaching.
- TCP/IP = practical & implemented.

OSI (7 Layers) Maps to TCP/IP (4 Layers)

7	Application	Application
6	Presentation	
5	Session	Transport
4	Transport	
3	Network	Internet
2	Data Link	Network Access (Link + Physical)
1	Physical	

1.2 Comparison: OSI vs TCP/IP

Feature	OSI Reference Model	TCP/IP Protocol Stack
Full Name	Open Systems Interconnection	Transmission Control Protocol / Internet Protocol
Number of Layers	7 Layers	4 Layers
Developed By	ISO (International Organization for Standardization)	DARPA (Defense Advanced Research Projects Agency)
Nature	Conceptual & theoretical mapping scheme	Working protocol stack implemented in hardware & software

Computer Networks Foundations - Lecture 04

Devices, Topologies & the Cloud Network

1. The Why of Network Devices

1.1 Why Direct Cables Do Not Scale

Connecting every pair of n devices with a dedicated cable $\rightarrow$

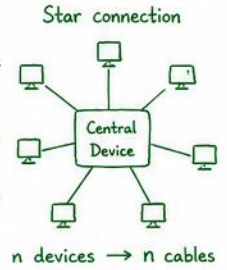
$$\text{Cables} = \frac{n(n-1)}{2}$$

- Cables grow quadratically ($\propto n^2$). Impossible to manage in offices or datacenters.

Devices (n)	Cables Needed
3	3
10	45
100	4,950
1,000	499,500

1.2 The Centralised Pivot Idea

- Connect every device to **ONE** central device.
- Only n cables required.
- This central pivot is the reason networking hardware exists.
- Devices differ by how deep they look into packets to make decisions.



2. Core Networking Devices

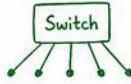
2.1 Hub (L1 Layer)

- Layer 1 (Physical) device.
- Replicates signal to **ALL** ports.
- No MAC/IP understanding.
- Weakness:** Unnecessary traffic $\rightarrow$ congestion as network grows.



2.2 Switch (L2 Layer)

- Layer 2 (Data Link) device.
- Learns MAC address $\leftrightarrow$ Port mapping.
- Unicasts frame to destination port.
- Eliminates local congestion.
- Cannot read IP $\rightarrow$ cannot forward to other networks.



2.3 Router (L3 Layer)

- Connects different networks.
- Forwards using **IP** addresses.
- Makes path-finding decisions (routing).



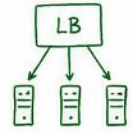
2.4 Firewall (L3/L4 Filtering)

- Security device.
- Inspects L3 (IP) + L4 (Port).
- Allows or blocks packets based on rules.



2.5 Load Balancer (L4/L7 Distribution)

- Distributes traffic across multiple servers.
- Works at L4 (Transport) or L7 (Application).
- Prevents overload, ensures high availability.

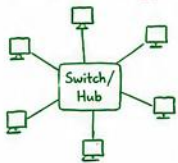


2.6 Core Devices Summary Table

Device	Layer	Decision Parameter	What it Does
Hub	L1 - Physical	Nothing (electrical)	Repeats signal to all ports
Switch	L2 - Data Link	MAC addresses	Forwards frame to correct port
Router	L3 - Network	IP addresses	Routes packets between networks
Firewall	L3/L4 (and above)	IP addresses & Port numbers	Filters traffic (allow/block)
Load Balancer	L4 / L7	Distribution logic	Distributes across server pool

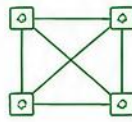
3. Network Topologies

3.1 Star Topology



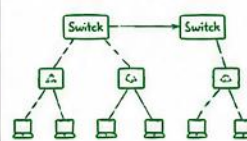
- All devices connect to central switch/hub.
- Easy to setup & maintain.
- Single point of failure (central device).

3.2 Mesh Topology



- Every device connects to every other device.
- High fault tolerance.
- Cable cost = $\frac{n(n-1)}{2}$
- Common in wireless (Wi-Fi mesh).

3.3 Hybrid Topology



- Combines two or more topologies.
- Real networks are almost always hybrids.

3.4 Environment Selection & Trade-offs

Environment	Suitable Topology	Primary Reason	Trade-offs
Small Office / LAN	Star	Easy installation & centralized maintenance.	Central point failure.
Smart Home	Mesh	Direct wireless device-to-device paths.	More complex, higher cost.
Enterprise Datacenter	Hybrid	Scaling flexibility + high reliability.	Design & management complexity.

4. Software-Defined Cloud Networks

4.1 The Evolutionary Shift

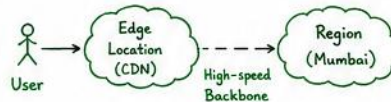
Same concepts implemented in software.

Physical Boxes $\rightarrow$ Virtual Instances
Copper Cables $\rightarrow$ Virtual Overlays
Manual Wiring $\rightarrow$ API calls & Console clicks

Concepts don't change, only implementation moves to software.

4.2 AWS Regions & Edge Caching

- Regions = geographic clusters (e.g., Mumbai).
- Connected by high-speed fiber backbones.
- Edge Locations (CDN) = cache close to users $\rightarrow$ reduces propagation delay (d in d/s).



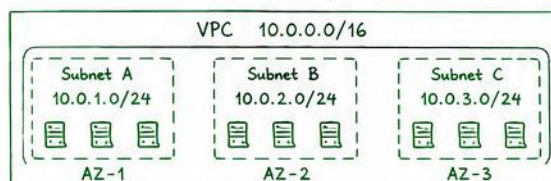
4.3 Redundancy: Availability Zones

- AZ = one or more isolated datacenters with redundant power, cooling, network.
- Deploy across multiple AZs $\rightarrow$ survive disasters (fire, flood, power failure).



4.4 Virtual Private Cloud (VPC) & Subnets

- VPC = private, logically isolated network in cloud.
- Control routing, security gates, IP address space.
- Network divided into Subnets mapped to AZs.
- IP allocation uses CIDR notation.



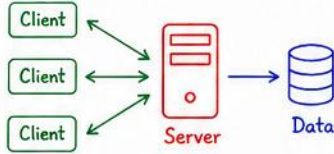
CIDR Cheat
/24 $\rightarrow$ 256 IPs
/16 $\rightarrow$ 65,536 IPs
/8 $\rightarrow$ 16,777,216 IPs

Lecture 05 - Application Layer: Architecture, Protocols and APIs

1 Network Architectures

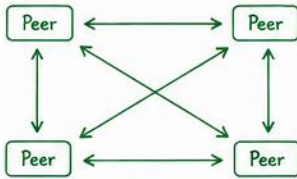
1.1 Client/Server Model

- Centralized: dedicated servers store data & process requests.
- Server resources scale for many clients.
- Clients do NOT communicate directly, all traffic goes via server.



1.2 Peer-to-Peer (P2P) Model

- Decentralized: every device (peer) acts as both client & server.
- Equal privileges & capabilities.
- No central server → highly resilient, no single point of failure.



2 The Application Layer & HTTP Basics

2.1 Layer 7 Role & Protocols

- Application Layer (L7) is interface between application software (browsers, email clients, games) & the network.
- Interprets user commands & manages data exchange using high-level protocols.

Common Application Layer Protocols

HTTP/HTTPS - Web browsing
SMTP - Email sending
FTP - File Transfer
DNS - Name Resolution

2.2 HTTP: Stateless Web Protocol

- Client (browser) sends request.
- Server processes & sends response.
- HTTP is stateless → server remembers nothing between requests; each request is independent.

2.3 Handling State: Cookies & Sessions

- Server creates a session & sends unique session key (Session ID) in response header (Set-Cookie).
- Browser stores cookie & automatically sends it in every subsequent request (Cookie header) → maintains user state.

3 HTTP Messages Breakdown

3.1 Request Breakdown (Line, Headers, Body)

```
GET /index.html HTTP/1.1 ← Request Line
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Content-Type: application/json ← Headers (Metadata)

{ "username": "john",
  "password": "1234" } ← Body (Optional)
```

3.2 Response Breakdown (Status, Headers, Body)

```
HTTP/1.1 200 OK ← Status Line
Content-Type: text/html
Content-Length: 1256
Server: nginx/1.20
Date: Mon, 26 Aug 2026 12:30:00 GMT ← Headers

<!DOCTYPE html>
<html> ... </html> ← Body (Resource content)
```

3.3 Standard HTTP Status Codes

- 200 OK : Request succeeded; resource returned.
- 301 Moved Permanently: Resource moved to new URL.
- 302 Found : Temporary redirect.
- 403 Forbidden : Access refused by server.
- 404 Not Found : Resource cannot be located.
- 500 Internal Server Error: Server crash/error.

4 HTTP Generations (1.0 to 3.0)

Version	Year	Key Features	Advantages	Limitations
4.1 HTTP/1.0 (Single TCP Connection)	1996	- New TCP connection for every single request. - Connection closed after response.	Simple to implement.	High overhead; slow loading for pages with many files.
4.2 HTTP/1.1 (Keep-Alive & FIFO blocking)	1997	- Persistent TCP connection (Keep-Alive). - Requests/responses in FIFO (sequential) order.	Reduces connection overhead; faster than 1.0.	Head-of-Line (HoL) blocking at application level → one slow resource blocks others.
4.3 HTTP/2 (Binary, Multiplexing & Server Push)	2015	- Binary framing layer. - Multiplexing: multiple streams concurrently over single TCP. - Server Push: server sends assets proactively.	Better performance, lower latency, compressed headers.	Runs on TCP → packet loss causes TCP-level HoL blocking (entire connection stalls).
4.4 HTTP/3 (QUIC over UDP & Stream Isolation)	2022	- Uses QUIC protocol over UDP. - Independent multiplexed streams. 0-RTT connection setup.	Solves HoL blocking; better performance, resilient to packet loss; faster connection setup.	Requires QUIC support; still newer in ecosystem (adoption ongoing).

Mnemonic: 1.0 = One file per connection | 1.1 = Keep connection | 2 = Multiplex | 3 = QUIC (Quick)

5 REST APIs & Inter-App Communication

5.1 REST Architectural Principles

- REST (Representational State Transfer) → most popular web architecture (~83% public APIs).
- Uses standard HTTP methods to operate on resources.
- Stateless (each request independent).
- Resource-based (identified by URLs).
- Standard data format → JSON (mostly).

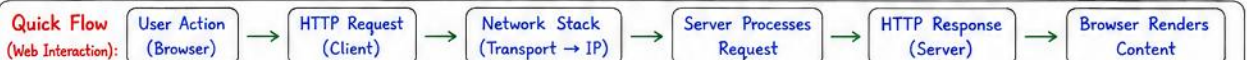
HTTP Methods in REST

GET	Retrieve resource
POST	Create new resource
PUT	Update entire resource
PATCH	Partial update
DELETE	Remove resource

Example: GET /users/42 → get user with ID 42

5.2 API Standardization via OpenAPI

- OpenAPI (Swagger) is an open specification to describe, design, document & test REST APIs.
- Written in JSON or YAML.
- Benefits:
 - ✓ Human-readable documentation
 - ✓ Machine-readable (code generation)
 - ✓ Client SDK generation
 - ✓ API testing & validation



Lecture 06 - HTTPS and The TLS Handshake: Securing the Web

① The Padlock & Plaintext Vulnerabilities

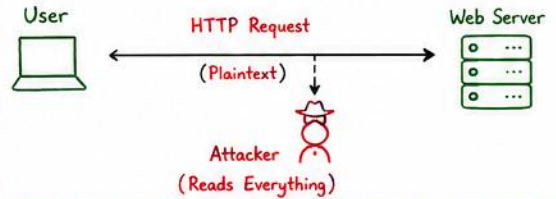
1.1 Browser Padlock Breakdown



- × Safe Website? → × No. Bad actors can buy domains & obtain lock icons too.
- ✓ Encrypted Connection? → ✓ Yes. Data cannot be read by middle hops.
- ✓ Verified Identity? → ✓ Yes. Site domain matches digital certificate identity.

1.2 Plaintext Eavesdropping Risk

HTTP is plaintext by design. Anyone capturing packets (attacker on public Wi-Fi / router) can read raw logins, credentials, messages.



② HTTPS & Cryptographic Core Goals

2.1 The HTTPS Definition

HTTPS is the secure version of HTTP. It wraps HTTP traffic inside an encrypted transport layer tunnel.

HTTPS = HTTP + TLS

2.2 Transition: SSL to TLS

Originally HTTPS used SSL (Secure Sockets Layer). Due to security flaws in SSL, modern systems use TLS (Transport Layer Security).

2.3 Core Goals: CIA

- **Confidentiality (Privacy):** Info readable only by authorized endpoints. (No eavesdropping)
- **Integrity:** Data remains unaltered & accurate. (No tampering)
- **Authentication:** Verifies identity of server (and sometimes client).

③ Symmetric vs. Asymmetric Encryption

3.1 Symmetric Key Exchange Issue

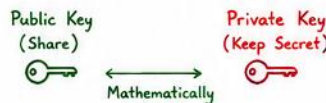
Uses single shared secret key for encryption + decryption.



Problem: How do strangers securely share K on an insecure internet without someone eavesdropping?

3.2 Asymmetric Public/Private Keys

Uses mathematically linked key pair.

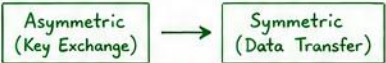


- Anyone can encrypt using Public Key.
- Only owner can decrypt using Private Key.

3.3 The TLS Hybrid Key Insight

TLS combines best of both worlds:

1. Uses **Asymmetric Encryption** initially to share a symmetric session key securely.
2. Switches immediately to **Symmetric Encryption** using that key for fast session communications.



④ The TLS Handshake Flow

4.1 Step 1: ClientHello

Client initiates handshake by sending:

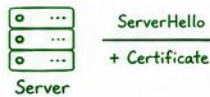
- Supported TLS versions
- Cipher suites (preferences)
- Random nonce (client_random)
- Other extensions



4.2 Step 2: ServerHello + Certificate

Server responds with:

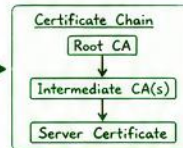
- Chosen TLS version
- Chosen cipher suite
- Random nonce (server_random)
- **Digital Certificate** (contains public key + CA signatures)



4.3 Step 3: Certificate Authentication

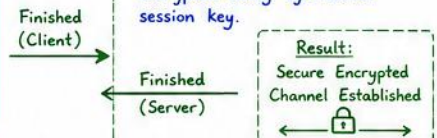
Client validates certificate chain using trusted CAs. If signature chain is valid, client trusts server identity.

Prevents Man-in-the-Middle (MitM) attacks.



4.4 Step 4: Key Exchange & Encryption Start

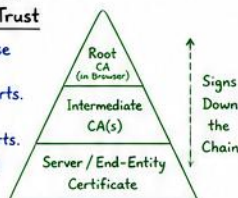
Both sides compute shared session key using exchanged nonces + key exchange method (e.g., (EC)DHE). They send Finished messages. From now on, all data is encrypted using symmetric session key.



⑤ Trust Chains & Cloud Deployments

5.1 Certificate Authorities & Chain of Trust

- Browsers trust digital certificates because they are signed by trusted CAs.
- Browsers ship pre-loaded with Root CA certs.
- Root CA signs Intermediate CA(s).
- Intermediate CA signs individual server certs.
- Browser validates full chain from end-entity cert → intermediate(s) → root.



If any signature fails → connection blocked.

Quick Recap

- Padlock = Encrypted + Verified Identity (not trust of website content)
- HTTPS = HTTP over TLS (secure transport)
- TLS Goal = Confidentiality + Integrity + Authentication (CIA)
- TLS Handshake = Negotiate → Authenticate → Exchange Keys → Encrypt

5.2 AWS Certificate Manager (ACM) Deployment

- Manual SSL/TLS cert management is tedious.
- AWS Certificate Manager (ACM) automates:
 - ✓ Certificate issuance (public / private)
 - ✓ Domain validation (DNS / Email)
 - ✓ Automatic renewal before expiry
- Integrates easily with:
 - CloudFront (CDN)
 - Application Load Balancer (ALB)
 - API Gateway, etc.



Important Terms

- CA = Certificate Authority
- Certificate = Public Key + Identity + CA Signature
- Nonce = Random number used once
- Cipher Suite = Algorithms for Key Exchange, Encryption, MAC, etc.

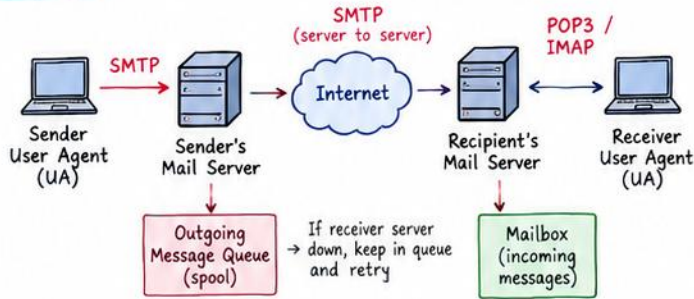
Exam Tip
Always draw TLS handshake steps in order (1 to 4)!

Electronic Mail Protocols

SMTP, POP3, IMAP & DNS MX

1 Email Architecture & Components

- User Agent (UA):**
 - Compose, read, reply, forward
 - Examples: Outlook, Gmail, Apple Mail
- Mail Server:**
 - Stores mailboxes
 - Manages incoming/outgoing queues
 - Executes routing protocols
- Spooling & Message Queue:**
 - If destination server is down, message is stored in queue and retried periodically (every 15-30 min for days)
- Asynchronous Delivery:**
 - Sender and receiver don't need to be online at the same time.



2 Simple Mail Transfer Protocol (SMTP)

- Used to transfer email messages across TCP connections (RFC 5321).
- Strictly a push protocol (client always pushes).
- Port 25** → server to server
- Port 587** → authenticated client submission
- Plain ASCII text commands and 3-digit response codes.

SMTP Command Sequence

HELO / EHLO → identify client
MAIL FROM: <sender@domain.com>
RCPT TO: <recipient@domain.com>
DATA → start message body (end with single dot . on a line)
QUIT → terminate session

Common SMTP Response Codes

220	Service Ready
250	Requested mail action okay / completed
354	Start mail input; end with <CR><LF>.<CR><LF>
550	Requested action not taken (e.g., mailbox unavailable)

Example SMTP Session (Simplified)

S: 220 mail.example.com Service Ready
 C: EHLO client.com
 S: 250 Hello client.com
 C: MAIL FROM: <alice@client.com>
 S: 250 OK
 C: RCPT TO: <bob@example.com>
 S: 250 OK
 C: DATA
 S: 354 End data with <CR><LF>.<CR><LF>
 C: (message body ...)
 S: 250 Message accepted
 C: QUIT
 S: 221 Bye

3 Mail Access Protocols: POP3 vs IMAP

- SMTP cannot deliver emails directly to user devices.
- Recipient's mail server stores messages until the user agent fetches them using a pull protocol (POP3 or IMAP).

POP3 (Post Office Protocol v3)

- Runs on TCP port 110 (or 995 over TLS)
- 3 phases: Authorization, Transaction, Update
- Download-and-Delete mode
- Download-and-Keep mode
- Simple, does not support folders or multi-device sync.

IMAP (Internet Message Access Protocol)

- Runs on TCP port 143 (or 993 over TLS)
- Maintains state on server
- Supports folders, flags (read/unread, starred, etc.)
- Full multi-device synchronization
- Header-only preview and selective fetch (bandwidth efficient)

POP3 vs IMAP Comparison

Feature	POP3	IMAP
Server State	Stateless across sessions	Stateful (folder tree & flags)
Multi-Device Sync	Poor / Unsynchronized	Full synchronization
Bandwidth Efficiency	Downloads full messages	Header-only preview & selective fetch
Offline Access	Full offline access (local copies)	Requires local caching config

4 MIME Extension & Binary Content Encoding

- Original SMTP supports only 7-bit ASCII text.
- MIME (RFC 2045) extends email headers to allow non-ASCII characters and binary attachments without changing SMTP.
- Used to declare content type and encoding methods.

Important MIME Header Lines

MIME-Version: 1.0
 Content-Type: multipart/mixed; boundary="====_NextPart"
 Content-Transfer-Encoding: base64

Base64 Encoding

- Encodes binary data into 7-bit ASCII strings.
- Groups 3 bytes (24 bits) → four 6-bit integers.
- Each 6-bit integer mapped to 64 ASCII chars [A-Z, a-z, 0-9, +, /].
- Introduces ~33% size expansion.



Quoted-Printable Encoding

- Used for mostly-text content with occasional non-ASCII bytes.
- Replaces non-ASCII byte with = followed by two hex digits.
- Example: € → =E2=82=AC

5 Email Infrastructure & Security

DNS MX (Mail Exchanger) Records

- To send email to user@example.com, the sender's mail server queries DNS for the MX record of example.com.
- DNS returns domain name and priority of destination mail server.
- Mail is then sent to the specified mail server.

DNS Query
(MX for example.com)



MX Records (example.com)
 10 mail1.example.com
 20 mail2.example.com

Email Spoofing

- SMTP MAIL FROM and header From: fields are plain strings, so attackers can easily forge sender addresses (phishing).

Authentication Frameworks: SPF, DKIM & DMARC

SPF (Sender Policy Framework)

- Domain owners publish a DNS TXT record listing all IP addresses authorized to send mail on behalf of their domain.

DKIM (DomainKeys Identified Mail)

- Outgoing mail servers add a cryptographic digital signature (RSA/Ed25519) to headers.
- Receivers verify using public key from DNS.

DMARC

- Combines SPF and DKIM validation.
- Allows domain owners to specify policy rules: none, quarantine, reject.

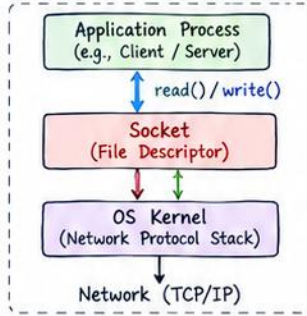
Lecture 08 The Socket API & Network Programming

1 Socket Abstraction & Transport Layer Interface

- A socket is an OS abstraction representing an end-point of communication between an application process and the network protocol stack.
- Developed in Berkeley (BSD Unix) in 1983.
- Socket API is the standard interface for network programming.

struct sockaddr_in (IPv4)

```
struct sockaddr_in {  
    sa_family_t    sin_family; // AF_INET for IPv4  
    in_port_t      sin_port;   // 16-bit Port (Network Byte Order)  
    struct in_addr sin_addr;    // 32-bit IPv4 Address  
};
```



File Descriptor Model

- In Unix-like OS, everything is a file.
- A created socket is assigned an integer File Descriptor (FD).
- We use standard system calls like `read()`, `write()`, `close()` on sockets.

Socket Address (IP, Port)

Socket Address = (IP Address, Port Number)

Key Fields:

- `sin_family` → Address family (AF_INET)
- `sin_port` → Port number (in network byte order)
- `sin_addr` → IP address (32-bit)

2 Byte Ordering & Network Data Representation

Big-Endian vs Little-Endian

Little-Endian
(x86, x86-64, ARM)

Memory address	
0x00	0x78
0x01	0x56
0x02	0x34
0x03	0x12

Least Significant Byte (LSB)
at lowest address

Big-Endian
(Network Order, SPARC, legacy MIPS)

Memory address	
0x00	0x12
0x01	0x34
0x02	0x56
0x03	0x78

Most Significant Byte (MSB)
at lowest address

Network Byte Order Standard

- To prevent data corruption between heterogeneous CPU architectures, the Internet Protocol suite mandates **Big-Endian** as the universal **Network Byte Order**.

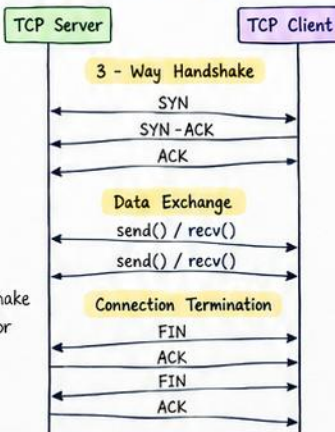
Endian Conversion Functions (POSIX)

<code>htons(uint16_t hostshort)</code>	→ Host to Network Short (16-bit)
<code>htonl(uint32_t hostlong)</code>	→ Host to Network Long (32-bit)
<code>ntohs(uint16_t netshort)</code>	→ Network to Host Short
<code>ntohl(uint32_t netlong)</code>	→ Network to Host Long

3 TCP Socket Lifecycle & Connection Workflow

Server (Passive Open)

- `socket(AF_INET, SOCK_STREAM, 0)`
→ Create a TCP socket
- `bind(sockfd, addr, len)`
→ Associate with local IP & port
- `listen(sockfd, backlog)`
→ Convert to passive mode
(backlog = max pending connections)
- `accept(sockfd, addr, len)`
→ Blocks until client completes 3-way handshake
→ Returns a new connected socket descriptor for that client



Client (Active Open)

- `socket(AF_INET, SOCK_STREAM, 0)`
→ Create a TCP socket
- `connect(sockfd, server_addr, len)`
→ Trigger 3-way TCP handshake (SYN, SYN-ACK, ACK)

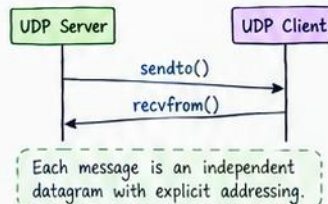
Data Exchange (TCP)

- TCP is a reliable byte-stream service.
- No message boundaries.
- Use `send()` and `recv()` to transmit raw bytes.
- `close(sockfd)` initiates 4-wave connection termination (FIN, ACK).

4 UDP Datagram Sockets & Connectionless I/O

UDP Overview

- Create socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
- UDP is **connectionless** → no handshakes.
- No `listen()` and no `accept()`.
- Suitable for applications like DNS, streaming, real-time data where low latency is important.



Datagram Transfer Functions

`sendto(sockfd, buf, len, flags, dest_addr, addrlen)`
→ Sends an independent datagram.

`recvfrom(sockfd, buf, len, flags, src_addr, addrlen)`
→ Receives a datagram and captures the sender's IP/port.

Message Boundaries

- UDP preserves message boundaries.
- A single `sendto()` call of 500 bytes results in exactly one `recvfrom()` call receiving 500 bytes (or truncating if buffer is small).

5 Advanced I/O Models & Multiplexing

Blocking vs Non-Blocking Sockets

- By default, socket operations (`accept()`, `recv()`, `connect()`) are **blocking** → thread sleeps until OS completes the operation or returns an error.
- Non-blocking sockets return immediately with an error (e.g., EAGAIN) if operation cannot be completed.

I/O Multiplexing

- Used to monitor multiple sockets for readiness to perform I/O.
- Event-driven model is more scalable for high-performance servers.

Scalability Limits

- Thread-per-connection model faces high context-switching and memory overheads when scaling to 10,000+ connections.
- Known as **The C10K Problem**.

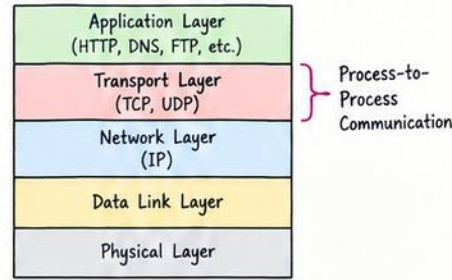
Mechanism	Key Points	Complexity / Limitations
<code>select()</code>	Monitors a set of file descriptors	O(N), limited to 1024 FDs
<code>poll()</code>	Removes 1024 cap	O(N), still requires array scanning & kernel-user copying
<code>epoll()</code>	Linux event-driven mechanism	O(1) event notification (used by NGINX, Node.js)

Lecture 09 The Transport Layer & User Datagram Protocol (UDP)

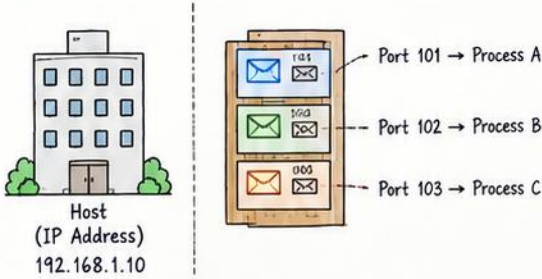
1 Transport Layer Basics

- The Transport Layer provides **logical communication** between application processes running on different network hosts.
- Network Layer: **host-to-host** (using IP addresses)
- Transport Layer: **process-to-process** (using port numbers)
- Adds transport header to application data and passes segments to the Network Layer.
- Performs multiplexing at sender and demultiplexing at receiver.

TCP/IP Stack (Position of Transport Layer)



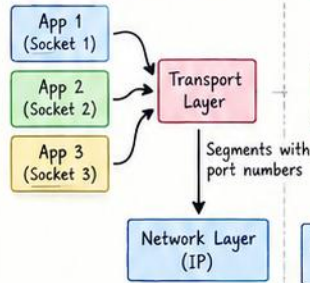
2 Analogy: Building & Mailboxes



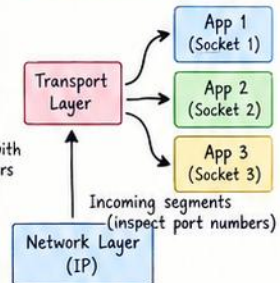
- Host IP address = Apartment building address
- Port number = Individual mailbox / apartment

3 Multiplexing & Demultiplexing

Multiplexing (Sender)



Demultiplexing (Receiver)



4 User Datagram Protocol (UDP)

- UDP (User Datagram Protocol) is a **minimal, connectionless** transport protocol (RFC 768).
- Provides only two added services:
 - Port-based demultiplexing
 - Lightweight error checking (checksum)
- No connection setup (no handshake).
- No connection state at endpoints.
- No congestion control.
- No guarantee of delivery or ordering.
- Gives direct access to the underlying IP layer.

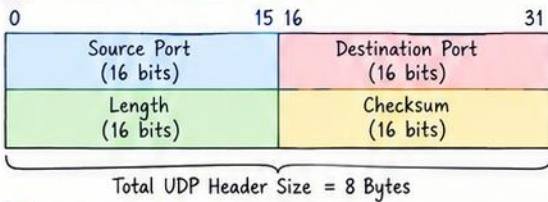
Key Characteristics of UDP

- Connectionless and stateless
- Low overhead (small 8-byte header)
- No 3-way handshake delay
- Application decides what and when to send
- Minimal server memory requirements
- Unreliable, unordered delivery
- Best-effort service

Why Developers Choose UDP?

- Fine-grained application control
- Zero connection setup overhead
- No connection state
- Small header overhead (8 bytes)
- Suitable for real-time and low-latency applications

5 UDP Header Format

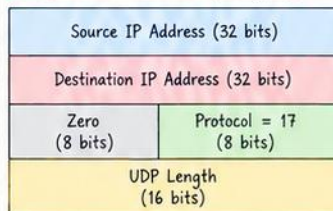


- Source Port (16 bits):** Port number of sender (can be 0 if not used).
- Destination Port (16 bits):** Port number of receiver.
- Length (16 bits):** Total length of UDP header + data (in bytes).
- Checksum (16 bits):** Error checking over pseudo-header, UDP header and data.

6 UDP Checksum Computation

- Build Pseudo-Header (12 bytes)**
 - Source IP address (32 bits)
 - Destination IP address (32 bits)
 - Zero (8 bits)
 - Protocol ID = 17 (8 bits)
 - UDP Length (16 bits)
- Combine:** Pseudo-header + UDP header + Data (treat as 16-bit words)
- Perform 1's complement summation** (carry overflow bits back into sum)
- Take 1's complement** (bitwise NOT) of result.

Pseudo-Header Format (12 bytes)



One's Complement Summation

- Add all 16-bit words.
- If carry occurs (overflow), add it back to the sum.
- After final sum, take 1's complement (bitwise NOT).

UDP Checksum

Detected errors in header, data and also IP addressing (via pseudo-header).

7 UDP Use Cases

- DNS: Uses UDP port 53 (request/response in single datagram).
- Real-time streaming & multimedia: RTP, VoIP, Zoom, WebRTC.
- Online gaming: Low latency is critical.
- HTTP/3 & QUIC: Modern web protocol built on top of UDP.
- IoT & sensor data: Lightweight, low overhead.
- Other: TFTP, SNMP, DHCP, NTP, etc.

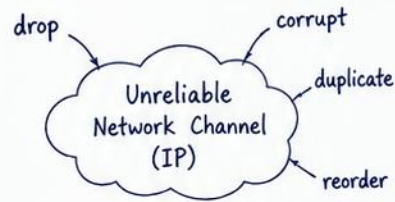
8 UDP vs TCP Comparison

Property	UDP	TCP
Connection State	Connectionless	Connection-oriented (3-way handshake)
Reliability	Unreliable (best-effort)	Reliable (ACKs & retransmissions)
Ordering	Unordered datagrams	Ordered byte-stream
Header Overhead	8 Bytes	20 - 60 Bytes
Congestion Control	None (sends at app rate)	Adaptive (AIMD, Slow Start)
Use Cases	DNS, VoIP, Streaming, Gaming, QUIC (HTTP/3)	Web browsing (HTTP/1.1, HTTP/2), Email, File transfer, etc.

Lecture 10 - Principles of Reliable Data Transfer (rdt)

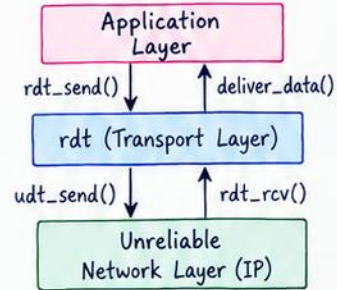
1 rdt Problem Framework

- Transport layer provides **Reliable Data Transfer (rdt)** service over an **unreliable Network Layer (IP)**
- Underlying channel can: drop packets, corrupt packets, duplicate packets, reorder packets



Layered Interface Primitives

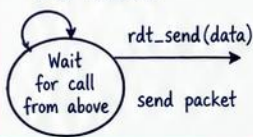
- rdt_send()**: called by application layer to pass data payload down
- udt_send()**: called by rdt to transmit packet over unreliable channel
- rdt_rcv()**: called by network layer when a packet arrives
- deliver_data()**: called by rdt to pass validated payload up to application



2 Building Blocks of Reliability: rdt 1.0 to rdt 3.0

rdt 1.0

- Assumes underlying channel is completely **reliable** (no bit errors, no lost packets)
- FSM needs only 1 state for sender and receiver



rdt 2.0

- Handles bit corruption
- Adds **Checksums**, **Positive ACKs** and **Negative ACKs (NAKs)**
- Flaw**: NAK/ACK corruption can cause duplicate packet delivery

rdt 2.1

- Adds 1-bit **Sequence Numbers** (0 or 1) to packets
- Receiver detects duplicate packets

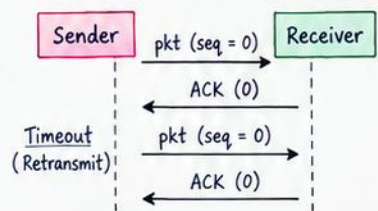
seq#	data	checksum
1 bit 0 or 1		

rdt 2.2

- Eliminates NAKs entirely
- Sends **duplicate ACKs** for the last correctly received packet sequence number

3 rdt 3.0: Handling Packet Loss with Countdown Timers

- rdt 3.0 (Alternating Bit Protocol)** handles packet loss.
- Introduces a countdown timer at sender.
- If ACK is not received within timeout period, sender retransmits packet.

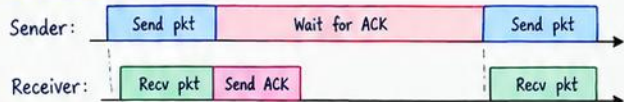


4 Stop-and-Wait Protocol & Utilization Bottleneck

Operational Flow

- Sender transmits 1 packet and stops
- Waits for receiver's ACK
- After receiving ACK, sends next packet

Timeline



Sender Channel Utilization

$$\text{Transmission Delay: } t_{\text{trans}} = \frac{L}{R}$$

$$U_{\text{sender}} = \frac{t_{\text{trans}}}{RTT + t_{\text{trans}}} = \frac{L/R}{RTT + L/R}$$

Example

- $R = 1 \text{ Gbps} = 10^9 \text{ b/s}$
- $RTT = 30 \text{ ms}$
- $L = 1 \text{ KB} = 8000 \text{ bits}$

$$t_{\text{trans}} = \frac{8000}{10^9} = 0.008 \text{ ms}$$

$$U_{\text{sender}} = \frac{0.008}{30 + 0.008} = 0.000267$$

Only 0.027% utilization!

5 Pipelined Protocols: Go-Back-N (GBN)

- Pipelining**: sender can have up to N unacknowledged packets in flight.
- Increases channel utilization by factor N .
- Sender window size = N .
- Receiver window size = 1 (accepts packets strictly in order).
- Cumulative ACK**: $\text{ACK}(n)$ indicates all packets up to and including sequence n have been received correctly.
- Single Timer**: tracks oldest unacked packet.
- On timeout, GBN retransmits all N unacked packets in current window.

6 Pipelined Protocols: Selective Repeat (SR)

- Each correctly received packet is individually acknowledged.
- Receiver maintains a buffer of size N to hold out-of-order packets.
- Avoids GBN's unnecessary retransmissions.

Sequence Number Space Rule

To avoid ambiguity when sequence numbers wrap around, window size N must satisfy:

$$N_{\text{sender}} + N_{\text{receiver}} \leq 2^k \Rightarrow N \leq 2^k - 1$$

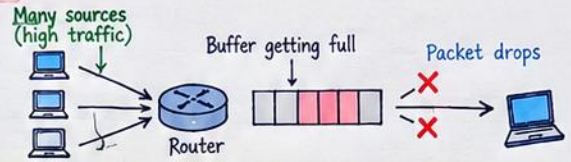
where k = number of bits in sequence number.

Lecture 11 - TCP Congestion Control & Flow Control

1. Network Congestion: Principles & Cost

What is Congestion?

Occurs when too many sources send data through a network link faster than routers can process and buffer it.



Causes of Congestion

- High traffic load (many sources)
- Limited link capacity
- Slow router processing
- Burstiness in traffic

Costs / Effects of Congestion

- High queueing delays (increased latency)
- Packet drops when buffers overflow
- Retransmissions of lost/delayed packets
- Wasted link capacity (spurious retransmissions)
- Degraded overall network performance

2. TCP Flow Control: Receive Window (rwnd)

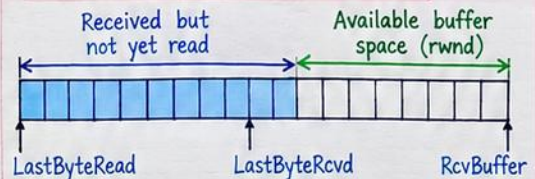
- Flow control prevents fast sender from overwhelming a slow receiving application's buffer
- End-to-end mechanism between sender and receiver (not network-wide)

Receive Window Formula

$$rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]$$

- Receiver advertises available buffer space in TCP header **Receive Window** field (rwnd).
- Sender must not send more than rwnd bytes beyond last acknowledged byte.

TCP Receive Buffer at Receiver



Flow Control vs Congestion Control

Flow Control

- Prevents fast sender from overwhelming receiving application's buffer.
- End-to-end mechanism.

Congestion Control

- Prevents sender traffic from overwhelming intermediate network router buffers.
- Network-wide effect.

3. TCP Congestion Control Principles

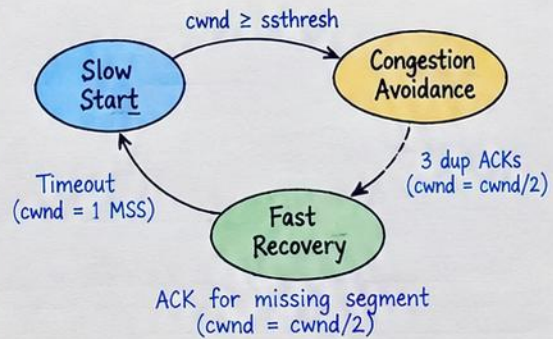
- Sender maintains **congestion window (cwnd)** limiting max unacknowledged data in flight.
- Actual data that can be sent is limited by both cwnd and rwnd.

$$LastByteSent - LastByteAcked \leq \min(cwnd, rwnd)$$

Effective Transmission Rate

- Roughly: $Rate \approx \frac{cwnd}{RTT}$ bytes/sec
- Larger cwnd → higher throughput
- cwnd adapts dynamically based on network conditions.

4. TCP Congestion Control State Machine



5. Additive Increase / Multiplicative Decrease (AIMD)

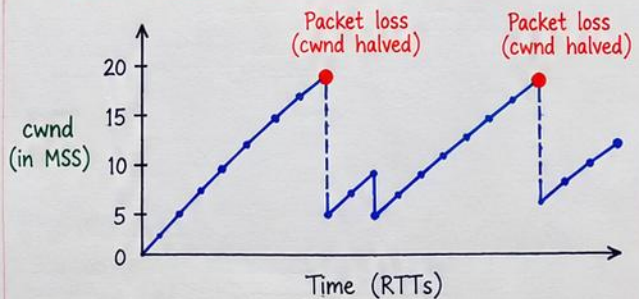
AIMD Principle

- TCP probes for available bandwidth by linearly increasing cwnd until loss occurs, then abruptly halving cwnd.
- Creates characteristic sawtooth wave pattern.

AIMD Rules

- **Additive Increase:**
 $cwnd = cwnd + 1 \text{ MSS}$
(for every RTT without loss)
- **Multiplicative Decrease:**
 $cwnd = cwnd / 2$
(upon detecting packet loss via 3 duplicate ACKs)

Typical cwnd Evolution (Sawtooth)



6. TCP Flavors & Modern Extensions

TCP Tahoe

- On any loss (Timeout OR 3 duplicate ACKs):
 - Resets cwnd = 1 MSS
 - Enters Slow Start
- More conservative, simpler behavior.

TCP Reno

- Distinguishes loss types:
 - **On 3 duplicate ACKs:**
 - Sets cwnd = cwnd / 2
 - Enters Fast Recovery (bypasses Slow Start)
 - **On Timeout:**
 - Resets cwnd = 1 MSS
 - Enters Slow Start

Fast Retransmit (3 Duplicate ACKs)

- When receiver gets 3 duplicate ACKs (indicating a missing segment), sender immediately retransmits the missing segment without waiting for the retransmission timer to expire.
- Reduces recovery time and improves throughput.

Cloud Load Balancing & Traffic Management

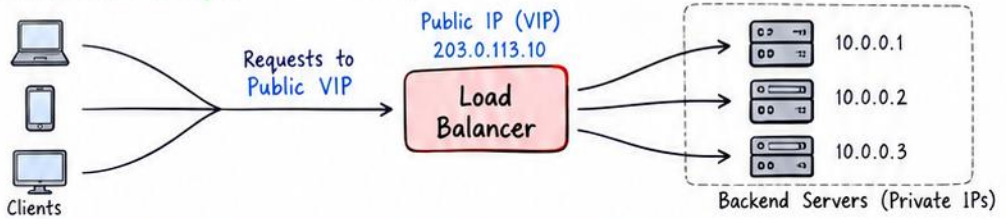
1 Load Balancing Fundamentals & Cloud Architecture

High Availability & Horizontal Scaling

- Load Balancer sits in front of server pool.
- Distributes incoming application traffic across multiple backend instances.
- Ensures **High Availability (HA)**.
- Allows **horizontal scaling** (add more servers).

Virtual IP (VIP) Abstraction

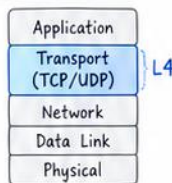
- Clients send requests to a single public **Virtual IP (VIP)**.
- Load Balancer proxies or forwards requests to internal **Private IPs (DIPs)** of backend servers.



2 Layer 4 (L4) vs. Layer 7 (L7) Load Balancing

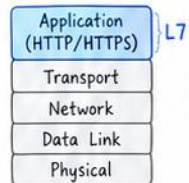
L4 Network/Transport Load Balancing

- Operates at **TCP/UDP** layer.
- Does not inspect application payloads.
- High throughput, low latency.
- Examples: AWS Network Load Balancer, IPVS, F5 (L4)



L7 Application Load Balancing

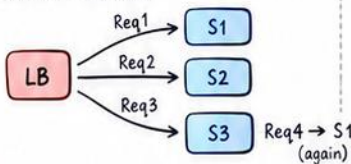
- Terminates TCP/TLS connections.
- Inspects HTTP headers, URLs, cookies, payloads etc.
- Routes traffic intelligently.
- Examples: NGINX, HAProxy, AWS ALB, F5 (L7)



3 Load Balancing Scheduling Algorithms

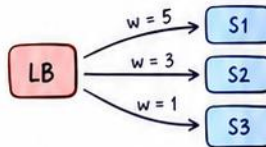
Round Robin

- Distributes requests sequentially across backend servers.
- Simple and effective for uniform servers.



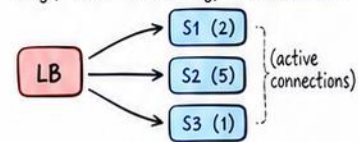
Weighted Round Robin

- Assigns higher traffic proportion to more powerful servers.
- Weight based distribution.



Least Connections

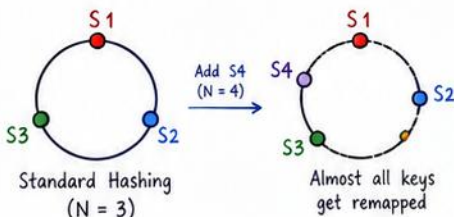
- Directs new requests to the server with fewest active connections.
- Useful for long-lived sessions (e.g., video streaming, WebSockets).



4 Consistent Hashing & Distributed Cache Distribution

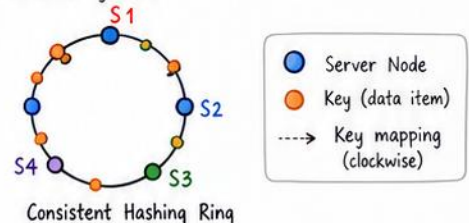
The Traditional Hash Problem

- Using standard modulo hashing:
$$\text{server} = \text{hash}(\text{key}) \bmod N$$
- When we add or remove a node ($N \rightarrow N+1$), nearly **100%** of keys get remapped, invalidating caches.



Hash Ring Topology (Consistent Hashing)

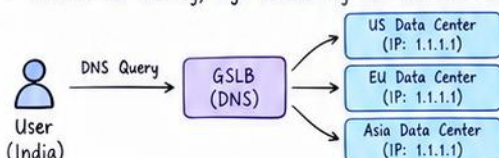
- Both servers and keys are mapped onto a circular $2^{32} - 1$ hash ring space.
- Adding or removing a server remaps only K/N keys on average (where K is total keys, N is number of servers).
- Minimizes cache invalidation and provides stability in distributed systems.



5 Global Server Load Balancing (GSLB) & DNS Routing

Multi-Region High Availability

- Global Server Load Balancing (GSLB) routes user traffic across geographically dispersed data centers worldwide.
- Ensures low latency, high availability and disaster recovery.



Anycast BGP Routing Mechanics

- In Anycast, multiple data centers announce the same IP address via BGP.
- Internet routers automatically direct user packets to the topologically closest data center.

Anycast - Same IP announced from multiple locations

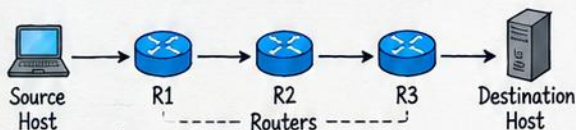


Addressing the World: IPv4 & Classless CIDR Routing

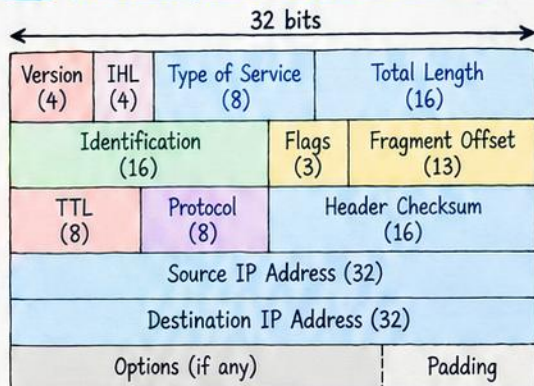
1 Network Layer Role & IPv4 Datagram Structure

1.1 Host-to-Host Delivery Model

- Network layer (IP) transports packets from a source host to a destination host across multiple routers.
- Two core functions:
 - Forwarding (local action)**: Move packet from an incoming link to correct outgoing link using forwarding table.
 - Routing (network-wide)**: Determine the path (packet route) from source to destination using routing protocols.



1.2 IPv4 Datagram (20-Byte Base Header)



Minimum header size = 20 bytes (without options)

2 IPv4 Addressing Fundamentals

- IPv4 address is a 32-bit integer.
- Written as 4 octets (8 bits each) separated by dots.
- Each octet value: 0 - 255.
- Dotted decimal notation: a.b.c.d

Total Address Space = $2^{32} = 4,294,967,296$ addresses (~4.3 billion)

Example

Binary: 11000000 10101000 00000001 00000001

192 - 168 - 1 - 1

Dotted decimal: 192.168.1.1

Key Facts

- 32-bit address
- 4 octets (8 bits each)
- Each octet: 0 - 255
- Example valid addresses:
 - 0.0.0.0
 - 192.168.1.1
 - 255.255.255.255

3 Historical Classful Addressing

Class	First Bits	Range (First Octet)	Network ID (netid)	Host ID (hostid)	Addresses per Network	Typical Use
Class A	0	0 - 127 (0.0.0.0 - 127.255.255.255)	8 bits	24 bits	16,777,216 (16M)	Very large networks
Class B	10	128 - 191 (128.0.0.0 - 191.255.255.255)	16 bits	16 bits	65,536 (65K)	Medium networks
Class C	110	192 - 223 (192.0.0.0 - 223.255.255.255)	24 bits	8 bits	256 (254 usable)	Small networks
Class D	1110	224 - 239	Multicast (no network/host division)			Multicast
Class E	1111	240 - 255	Reserved for experimental use			Experimental

4 Classless Inter-Domain Routing (CIDR)

- CIDR (RFC 1519) replaced rigid classful system.
- Allows variable-length network prefixes.
- Address format: a.b.c.d/x (x = number of prefix bits).
- First x bits → network prefix, remaining (32 - x) bits → host bits.

4.1 CIDR Notation (/x)

Example: 192.168.1.0/24

- /24 → 24-bit network prefix
- Remaining 8 bits → host bits
- Different subnets possible (VLSM).

4.2 Subnet Mask Representation

- Subnet mask has x leading 1s followed by (32 - x) 0s.
- Converts /x to dotted decimal.

Prefix (/x)	Subnet Mask
/8	255.0.0.0
/16	255.255.0.0
/24	255.255.255.0
/30	255.255.255.252
/32	255.255.255.255

4.3 Longest Prefix Match Rule

- When forwarding a packet, router searches routing table for all prefixes that match the destination IP.
- Selects the entry with the **longest prefix length (largest x)**.
- Most specific match wins.
- If no match found, packet is sent to default route (0.0.0.0/0).

Example:

192.168.0.0/16 → match

192.168.1.0/24 → longest match (selected)

5 Special IPv4 Address Ranges

5.1 Private IP Space (RFC 1918)

- Used in private networks (LANs).
- Not routable on the public Internet.

Private Block	Range
10.0.0.0/8	10.0.0.0 - 10.255.255.255
172.16.0.0/12	172.16.0.0 - 172.31.255.255
192.168.0.0/16	192.168.0.0 - 192.168.255.255

5.2 Other Important IPv4 Ranges

Address / Range	Purpose
0.0.0.0	Unspecified address (boot/initialization)
127.0.0.0/8	Loopback (e.g., 127.0.0.1)
169.254.0.0/16	Link-local (APIPA)
224.0.0.0/4	Multicast (Class D)
240.0.0.0/4	Reserved (Class E)
255.255.255.255	Limited broadcast

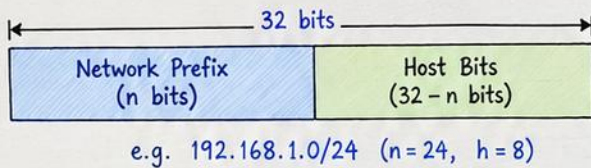
5.3 Key Takeaways

- IPv4 uses 32-bit addresses (4 octets in dotted decimal).
- CIDR uses flexible /x prefix instead of fixed classes.
- Subnet mask defines network and host portions.
- Routers use longest prefix match.
- Private IP ranges are for internal networks and not routed on Internet.

Subnetting in Practice & Network Address Translation (NAT)

① IP Addressing Recap

- IPv4 address = **32 bits** (4 octets)
e.g. 192.168.1.10 (written in dotted decimal)
- Format: **<Network Prefix> / <Prefix Length>**
e.g. 192.168.1.0/24
- Subnet mask separates network and host bits.
- Network ID**: first address (all host bits 0)
- Broadcast**: last address (all host bits 1)
- Usable hosts per network** = $2^h - 2$
(excluding network ID & broadcast)
h = number of host bits



③ Subnet Design Example

Divide 192.168.1.0/24 into 4 subnets (b = 2 bits)

- Original mask: /24 (255.255.255.0)
- New mask: /26 (255.255.255.192)
- Subnet size: $2^6 = 64$ addresses (62 usable)
- Total subnets: $2^2 = 4$

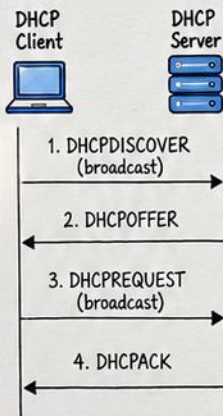
Subnet	Network Address (/26)	Usable Host Range	Broadcast
0	192.168.1.0	192.168.1.1 – 192.168.1.62	192.168.1.63
1	192.168.1.64	192.168.1.65 – 192.168.1.126	192.168.1.127
2	192.168.1.128	192.168.1.129 – 192.168.1.190	192.168.1.191
3	192.168.1.192	192.168.1.193 – 192.168.1.254	192.168.1.255

⑤ Dynamic Host Configuration Protocol (DHCP)

- Automatically assigns IP addresses, subnet masks, default gateways and DNS servers to hosts.
- Defined in RFC 2131.
- Uses **UDP ports 67** (server) and **68** (client).

DHCP DORA Exchange

- Discover**: Client broadcasts DHCPDISCOVER to 255.255.255.255 (UDP 68 → 67).
- Offer**: Server responds with DHCPOFFER proposing an IP lease.
- Request**: Client broadcasts DHCPREQUEST accepting the lease.
- ACK**: Server confirms with DHCPACK. Client can now use the IP.



② Subnetting Mathematics

- To divide a network into 2^b subnets, borrow b host bits for the network prefix.

$$\text{Number of Subnets} = 2^b$$

$$\text{Usable Hosts per Subnet} = 2^{32-(x+b)} - 2$$

- x = original prefix length
- New prefix length = x + b
- Subnet mask changes accordingly.

Borrowed Bits (b)	Subnets	Usable Hosts per Subnet
1	2	126
2	4	62
3	8	30
4	16	14

④ Variable Length Subnet Masking (VLSM)

- VLSM allows creating subnets of different sizes.
- Used to efficiently allocate IP addresses based on actual host requirements.
- Common in enterprise networks.

Example:

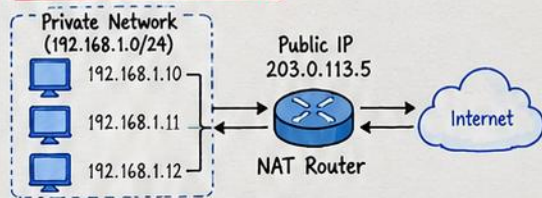
Given 192.168.10.0/24, allocate subnets for different departments.

Dept.	Required Hosts	Subnet	Usable Range	Mask
A (Admin)	50	192.168.10.0	.1 – .62	/26
B (HR)	25	192.168.10.64	.65 – .94	/27
C (R&D)	10	192.168.10.96	.97 – .110	/28
D (Sales)	5	192.168.10.112	.113 – .118	/29

⑥ Network Address Translation (NAT)

- Allows multiple private IP addresses to share a single public IP address on the Internet.
- Defined in **RFC 3022**.
- Hides internal network structure and conserves IPv4 addresses.
- Common in home routers and enterprise gateways.

NAT / NATP Working



NAT Translation Table (NAPT)

Private IP	Private Port	Public IP	Public Port	Destination
192.168.1.10	52341	203.0.113.5	40001	8.8.8.8:53
192.168.1.11	53421	203.0.113.5	40002	1.1.1.1:443
192.168.1.12	49123	203.0.113.5	40003	142.250.72.14:443

⑦ Types of NAT

Type	Description	Use Case
Static NAT	One-to-one mapping between private and public IP.	Servers accessible from Internet.
Dynamic NAT	Maps private IP to a pool of public IPs.	Limited public IP pool.
PAT (NAPT)	Many private IPs share one public IP using different ports.	Most common (home routers).

⑧ IPv6 Transition & Future Addressing

- IPv6 address: 128 bits → $2^{128} \approx 3.4 \times 10^{38}$ addresses.
- Written in **hexadecimal**, 8 groups of 4 digits.
- Example: 2001:0db8:85a3::8a2e:0370:7334

Global Routing Prefix (48 bits)	Subnet ID (16 bits)	Interface ID (64 bits)
---------------------------------	---------------------	------------------------

- Simplifies router processing (no header checksum, no fragmentation in base header).
- Transition strategies: Dual Stack, Tunneling, NAT64/DNS64.